Exercise 1.1: Paper Reviews

Evolution of the Graphics Processing Unit (GPU)

W. J. Dally, S. W. Keckler, and D. B. Kirk. *IEEE Micro*, vol. 41, no. 6, pp. 42–51, 2021.

1. Primary contribution

This article provides a historical synthesis of how GPUs evolved from fixed-function graphics hardware into programmable platforms for graphics, high-performance computing, and deep learning. The authors connect major developments, including programmable shaders, CUDA-enabled computation, and Tensor Cores, to the requirements of these applications. By examining hardware alongside programming tools and libraries, they explain how architectural capabilities and a supporting software ecosystem jointly enabled the adoption of GPUs beyond graphics.

2. Key insight of the contribution

The key insight is that GPU progress is driven by feedback between applications and architecture: programmability attracts new workloads, whose requirements then motivate specialized hardware. Tensor Cores illustrate this process by accelerating matrix multiply-and-accumulate operations and amortizing instruction overhead across many arithmetic operations. Together with suitable reduced-precision formats and software support, such specialization explains why the inference-performance trend called Huang's law exceeds the gains attributable to transistor scaling alone.

3. Opinion and reaction

I found it interesting that the drive for better video games helped create the computing platform behind the modern AI boom. The article presents this progression logically and makes the industry's direction understandable without relying on excessively detailed technical explanations. I also appreciated its emphasis on software, which makes clear why computational power alone is not enough to make a platform useful.

A Bridging Model for Parallel Computation

Leslie G. Valiant. *Communications of the ACM*, vol. 33, no. 8, pp. 103–111, 1990.

1. Primary contribution

Valiant introduces the bulk-synchronous parallel (BSP) model as a common framework between parallel software and hardware, with a role similar to that of the von Neumann model in sequential computing. The model combines processing and memory components, point-to-point communication, and controlled synchronization in units called supersteps. The paper supports this proposal with results on implementing shared-memory features and parallel algorithms on BSP, and on implementing BSP efficiently using different communication networks.

2. Key insight of the contribution

The key insight is that a program should expose more parallel work than the physical processors can execute at once. This parallel slackness gives the compiler room to distribute work, pipeline communication, and absorb uneven memory-access loads. Together with synchronization at a suitable granularity, it allows many low-level details to be managed automatically with only a constant-factor efficiency loss, provided enough parallelism and communication capacity are available.

3. Opinion and reaction

I found it interesting that the paper treats a shared model for software and hardware as an essential part of making parallel computing practical. The idea of exposing more parallelism than the machine can execute at once seems counterintuitive at first, but its role in handling communication and uneven workloads makes the argument convincing. I would have liked a more detailed programming example, since the efficiency arguments are harder to connect to an implementation than the basic model itself.